TDDE31-Reporrt1

Ahmad Soltani & Fangyun Yang

April 2026

1 Kernel Sum:

Predicted Temperatures (Kernel Sum) for Linköping in "2013-07-04"

The physical distance and temporal day distance are not very far away from the target place and date.

- $h_{\text{distance}} = 100$
- $h_{\text{date}} = 14$
- $h_{\text{time}} = 2$

Predicted Temp	Time
4.37	24:00:00
4.75	22:00:00
5.17	20:00:00
5.52	18:00:00
6.13	16:00:00
6.58	14:00:00
6.57	12:00:00
6.14	10:00:00
5.20	08:00:00
4.31	06:00:00
4.16	04:00:00

The physical distance and temporal day distance are far away from the target place and date.

- $h_{\text{distance}} = 1000$
- $h_{\text{date}} = 30$
- $h_{\text{time}} = 2$

Predicted Temp	Time
4.93	24:00:00
5.01	22:00:00
5.10	20:00:00
5.20	18:00:00
5.33	16:00:00
5.43	14:00:00
5.47	12:00:00
5.33	10:00:00
5.11	08:00:00
4.87	06:00:00
4.86	04:00:00

2 Kernel Product:

Predicted Temperatures (Kernel Product) for Linköping in "2013-07-04"

The physical distance and temporal day distance are not very far away from the target place and date.

- $h_{\text{distance}} = 100$
- $h_{\text{date}} = 14$
- $h_{\text{time}} = 2$

Predicted Temp	Time
12.28	24:00:00
13.24	22:00:00
14.83	20:00:00
16.43	18:00:00
17.42	16:00:00
17.86	14:00:00
17.65	12:00:00
16.93	10:00:00
15.66	08:00:00
14.19	06:00:00
12.69	04:00:00

The physical distance and temporal day distance are far away from the target place and date.

- $h_{\text{distance}} = 1000$
- $h_{\text{date}} = 30$

- $h_{\text{time}} = 2$

Predicted Temp	Time
10.95	24:00:00
11.86	22:00:00
13.38	20:00:00
14.96	18:00:00
15.95	16:00:00
16.40	14:00:00
16.28	12:00:00
15.63	10:00:00
14.44	08:00:00
13.02	06:00:00
11.54	04:00:00

3 Questions:

1. **Show that your choice for the kernels' width is sensible, i.e. it gives more weight to closer points. Discuss why your definition of closeness is reasonable.**

   Both the multiplicative and additive kernels are sensitive to the spatial distance and the temporal distance (denoted as `days_distance`) from the target location and date. As shown in the result tables, for $h_{\text{distance}} = 1000$ and $h_{\text{date}} = 30$, the predicted temperatures are lower. This behavior is due to the exponential function in the kernel: as the spatial and temporal distances from the target increase, the kernel values decrease exponentially. The points that are farther away contribute less to the prediction. Therefore, defining closeness in terms of both physical distance and time difference is reasonable, since nearby locations and recent observations are more relevant for estimating the target value.

2. **Repeat the exercise using a kernel that is the product of the three Gaussian kernels above. Compare the results with those obtained for the additive kernel. If they differ, explain why.**

   When switching to the multiplicative kernel, we obtain significantly different and generally higher predicted temperatures. This occurs because the multiplicative kernel acts as a stricter filtering mechanism, similar to a logical `AND` operation. Only data points that are simultaneously close in all dimensions (spatial and temporal) retain significant weight, while others are heavily down-weighted. In contrast, the additive kernel behaves more like a logical `OR`, allowing points that are close in at least one dimension to still contribute meaningfully. As a result, the multiplicative kernel emphasizes a smaller subset of highly relevant points, which explains the observed differences in predictions.

4 Code:

```
###BDA3_demo.py

from __future__ import division
from math import radians, cos, sin, asin, sqrt, exp
from datetime import datetime
from pyspark import SparkContext

sc = SparkContext(appName="lab_kernel")

def haversine(lon1, lat1, lon2, lat2):
    """
    Calculate the great circle distance between two points
    on the earth (specified in decimal degrees)
    """
    # convert decimal degrees to radians
    lon1, lat1, lon2, lat2 = map(radians, [lon1, lat1, lon2, lat2])
    # haversine formula
    dlon = lon2 - lon1
    dlat = lat2 - lat1
    a = sin(dlat/2)**2 + cos(lat1) * cos(lat2) * sin(dlon/2)**2
    c = 2 * asin(sqrt(a))
    km = 6367 * c
    return km

h_distance = 100
h_date     = 14
h_time     = 2

# h_distance = 1000
# h_date     = 30
# h_time     = 2

a    = 58.4274  # linköpings latitude
b    = 14.826   # linköpings longitude
target_date = "2013-07-04"  # the date for which we want to find the nearby stations and the

stations = sc.textFile("hdfs:///user/x_ahmso/BDA/input/stations.csv")
temps    = sc.textFile("hdfs:///user/x_ahmso/BDA/input/temperature-readings.csv")
#temps    = sc.textFile("hdfs:///user/x_ahmso/BDA/input/temperature-readings-small.csv")
# Your code here
lines_stations = stations.map(lambda line: line.split(";"))
lines_temps    = temps.map(lambda line: line.split(";"))

structured_stations = lines_stations.map(lambda x: (x[0], (float(x[3]), float(x[4])))) # (s
structured_temps    = lines_temps.map(lambda x: (x[0], x[1], x[2], float(x[3]))) # (station
```

```
stations_dict = sc.broadcast(structured_stations.collectAsMap()) # broadcast the station in

valid_temps = structured_temps.filter(lambda x: x[1] <= target_date)
valid_temps.cache()

results = []
for time in ["24:00:00", "22:00:00", "20:00:00", "18:00:00", "16:00:00",
             "14:00:00", "12:00:00", "10:00:00", "08:00:00", "06:00:00", "04:00:00"]:

    target_hour = int(time.split(":")[0])
    current_valid_temps = valid_temps.filter(lambda x: False if (x[1] == target_date and x[
    )

    def compute_weight(reading):
        stationID, date, time, temperature = reading

        data_lot, data_lon = stations_dict.value[stationID]
        dist_km = haversine(b, a, data_lon, data_lot)

        diff_days = (abs((datetime.strptime(target_date, "%Y-%m-%d") - datetime.strptime(dat
        dist_date = min(diff_days, 365 - diff_days)

        diff_hours = (abs(int(target_hour) - int(time.split(":")[0])))
        dist_time = min(diff_hours, 24 - diff_hours)

        kernel_distance = exp(-(dist_km**2) / (h_distance**2))
        kernel_date = exp(-(dist_date**2) / (h_date**2))
        kernel_time = exp(-(dist_time**2) / (h_time**2))

        total_weight_sum = kernel_distance + kernel_date + kernel_time
        #total_weight_product = kernel_distance * kernel_date * kernel_time

        return (total_weight_sum, total_weight_sum*temperature)
        #return (total_weight_product, total_weight_product*temperature)

    mapped_rdd = current_valid_temps.map(compute_weight)

    weighted_sum = mapped_rdd.reduce(lambda x, y: (x[0] + y[0], x[1] + y[1]))

    sum_weights = weighted_sum[0]
    sum_weighted_temps = weighted_sum[1]

    estimated_temp = sum_weighted_temps / sum_weights

    results.append(f"Estimated temperature for {time}: {estimated_temp}")
```

```
results_rdd = sc.parallelize(results)
results_rdd.saveAsTextFile("hdfs:///user/x_ahmso/BDA/output")


###run_yarn.q

#!/bin/bash
#SBATCH --time=30:00
#SBATCH --nodes=2
#SBATCH --exclusive

echo "START AT: $(date)"

module load spark/3.5.1-hadoop-3.3.6-hpc1-bdist

# Cleanup and start from scratch
rm -rf spark

# Startup hadoop filesystem and yarn
hadoop_setup

echo "Prepare output and input directories and files..."
# The following command will make folders on your home folder on HDFS, the input and output
hadoop fs -mkdir -p "/user/x_ahmso/BDA" "/user/x_ahmso/BDA/input"
hadoop fs -test -d "/user/x_ahmso/BDA/output"
if [ "$?" == "0" ]; then
    hadoop fs -rm -r "/user/x_ahmso/BDA/output"
fi

#hadoop fs -copyFromLocal ./input_data/temperature-readings-small.csv "/user/x_ahmso/BDA/inp
# Remove the comment when you need specifc file below
hadoop fs -copyFromLocal ./input_data/temperature-readings.csv "/user/x_ahmso/BDA/input/"
#hadoop fs -copyFromLocal ./input_data/precipitation-readings.csv "/user/x_ahmso/BDA/input/
hadoop fs -copyFromLocal ./input_data/stations.csv "/user/x_ahmso/BDA/input/"
#hadoop fs -copyFromLocal ./input_data/stations-Ostergotland.csv "/user/x_ahmso/BDA/input/"

# Run your program
echo "Running Your program..."

# spark-submit comes here and you need to replace "BDA1_demo.py" when you have/run your own
spark-submit --deploy-mode cluster --master yarn --num-executors 9 --driver-memory 2g --exec

echo "================= FINAL OUTPUT ========================================="
hadoop fs -cat "/user/x_ahmso/BDA/output"/*

rm -rf output
```

```
hadoop fs -copyToLocal '/user/x_ahmso/BDA/output' ./
hadoop_stop

echo "END AT: $(date)"
```